

RANGKUMAN PRAKTIS BASIS DATA

IF2240 - Basis Data

Jumat, 1 Mei 2020

Outline Materi UAS Basis Data:

- Review Materi Pra-UTS ✓
- Transform ER to Relational Model ✓
- Relational Database Design (Normalisasi) ✓
- Integrity Constraint ✓
- Storage & File Structure ✓
- Index & Schema Tuning ✓



Latman Collection Basis Data : [Day 5 - Basis Data](#)

Pembahasan UAS Basis Data 2019 (on progress) : [Pembahasan UAS Basis Data 2019](#)

Teknis Pengisian:

Pembuatan rangkuman merupakan salah satu atau lebih dari hal-hal berikut.

- Konsep atau inti pembahasan
- Maksud dari setiap materi / persoalan
- Langkah / algoritma yang diperlukan (seperti kapan dikatakan 3NF, cara dekomposisi)
- Teknik cepat dalam memahami suatu materi (recommended jika ada)
- Dilengkapi dengan satu buah contoh soal (jika memungkinkan atau sempat)
- Bila ada pertanyaan / salah ketik / ambigu untuk dibahas, gunakan fitur Add Comment seperti di samping, kalau dirasa yakin salah ketik, langsung benerin aja

Pelaksanaan Ujian Basis Data:

Hari / Tanggal : Senin, 4 Mei 2020
Waktu : 12:30 - 15:30 WIB
Durasi : 180 menit
Sifat Ujian : Individual, Open-Book

Saat ujian berlangsung:

1. Rangkuman akan **dikunci** menjadi mode view selama ujian berlangsung.
2. Kerjakan ujian segepat mungkin, karena waktu yang begitu terbatas dan soal yang sebagian besar 'mikir'.
3. Usahakan untuk mengerjakan literally "individual", sehingga perlu untuk belajar materi ini lebih matang.

Caution:

- Dilarang keras untuk menyebarkan rangkuman ini ke grup-grup selain IF'18 dan kawan-kawannya.
- Dilarang keras menyebarkan ke grup yang terdapat dosen dari bidang manapun.

A. Review Materi Pra-UTS

Difficulty : Medium

Credit : Mario Gunawan

1) Query MySQL

Diberikan sebuah basis data dengan relasi-relasi sebagai berikut.

- agencydb = {id, nama, umur, jenis_kelamin}
- agencyinfo= {id, notelp}

Query query penting:

a) SELECT a, b, c FROM e WHERE a = "hohoho" :

Penjelasan : memilih data a, b, c dari tabel e dengan kondisi a = "hohoho".

b) FROM agencydb NATURAL JOIN agencyinfo ...

Penjelasan : menggabungkan tabel agencydb dan agencyinfo berdasarkan nama kolom yang sama secara otomatis, dalam kasus ini id.

Ada 3 tipe join :

A. NATURAL JOIN

Menggabungkan 2 tabel berdasarkan kolom yang namanya sama. Misalnya ada suatu row di tabel pertama ataupun kedua yang gapunya pasangan row yang nilainya sama, maka tidak dimasukkan ke dalam relasi hasil. INGAT : natural join boleh ada null value, asalkan memang ada row di salah satu tabel yang datanya null (misal agencyinfo ada data (1, NULL), maka misal kita pake natural join antara agencyinfo dan agencydb, maka tetap bakal keluar row dengan id = 1 , tapi no telpnya null. Jadi yg ga dimasukin tuh misal memang gaada row yang matching)

B. LEFT/ RIGHT JOIN

Menggabungkan 2 tabel berdasarkan kolom yang dispesifikasi dengan syntax ON (nama tabel1).(namakolom1) = (namatabel2).(namakolom2), lalu misal left join, maka tabel yang di sebelah kiri join HARUS ADA SEMUA dalam relasi yang dihasilkan jadi yang kanan boleh null. Begitu juga right join, tp dibalik (wadigidaw). Contoh query:

... FROM agencydb LEFT JOIN agencyinfo ON agencydb.id = agencyinfo.id ...;

Ini berarti agencydbnya semua rownya harus termasuk dalam relasi hasil. LINK :

<https://www.mysqltutorial.org/mysql-left-join.aspx>

C. INNER JOIN

Menggabungkan 2 tabel berdasarkan kolom yang dispesifikkan kyk left sama right join, dengan mekanisme pencocokan yang sama kyk natural join (kalo gaada pasangan, dieliminasi rownya :()

(for visual learners: [click here](#))

- c) INSERT INTO agencydb values(1,"hehe",12,"apache helicopter");

Penjelasan : Menambahkan data baru dengan value tertentu

id auto increment INSERT INTO agencydb(nama,umur,jenis_kelamin)
values("hehe",12,"apache helicopter")

Penjelasan : Menambahkan data baru dengan incremental id

- d) UPDATE agencydb SET nama = "hans ganteng" WHERE id = 1;

Penjelasan : Mengupdate nama pada value dengan id = 1 dengan "hans ganteng".

- e) SELECT MAX(umur) FROM agencydb GROUP BY jenis_kelamin;

Penjelasan : Mencari umur maksimum dari agencydb dari masing masing jenis kelamin. MAX() merupakan fungsi agregat yang mengembalikan satu baris tiap elemen kolom di group by, walaupun nilai maksimumnya ada 2. Contoh fungsi agregat lain:

- COUNT() ⇒ menghitung banyaknya row
- MIN() ⇒ nilai minimum
- AVG() ⇒ nilai rata-rata
- SUM(kolom int) ⇒ jumlah suatu kolom (harus integer kolomnya, kalo string returnnya 0)

- f) Persoalan : mendapatkan nama-nama orang yang rata-rata umurnya lebih tinggi:

INNER QUERY

SELECT nama FROM agencydb WHERE umur > (SELECT AVG(umur) FROM agencydb);

Pada query ini, karena AVG(umur) hanya mereturn 1 kolom, maka bisa dibandingkan dengan kolom yang ada dalam agencydb.

- g) SET OPERATION

Set operation adalah sebutan untuk operasi antara 2 relasi atau tabel. Ada 3 operation set yang di mention di slide basis data:

UNION : ada di tabel 1 atau 2

INTERSECT : ada di tabel 1 dan 2

EXCEPT : ada di tabel 1 dan tidak ada di tabel 2

Jawaban soal UAS tahun 2016 no 1 (tentang query mysql) :

Soal :

1. Diberikan skema basis data akademik berikut ini.

MAHASISWA = (NIM, NAMA, TGL_LAHIR, PRODI, FAKULTAS)

KULIAH = (KODE, NAMA_KUL, SKS, PRODIKUL)

PERKULIAHAN = (NIM, KODE, SEMESTER, TAHUN, NILAI) → saat pencatatan awal, Nilai akan bernilai NULL. Nilai sebenarnya baru diisikan di akhir semester.

Tuliskan perintah *query* SQL untuk melakukan hal berikut.

- Daftar kode dan nama mata kuliah, besar SKS, dan nilai yang diperoleh oleh mahasiswa dengan NIM 13514999, hanya untuk pengambilan kuliah yang sudah ada nilainya, terurut berdasarkan SKS (mengecil) dan kode kuliah (membesar).
- Daftar kode, nama, dan besar SKS mata kuliah yang ditawarkan di semester 2 tahun 2015/2016, berikut jumlah mahasiswa yang mengikuti kuliah tersebut.
- Daftar nama dan prodi mahasiswa yang sudah mengambil semua mata kuliah yang ditawarkan oleh program studinya.
- Memeriksa apakah *functional dependency* PRODI → FAKULTAS terdefinisi pada relasi MAHASISWA. Jelaskan juga bagaimana hasil dari *query* tersebut digunakan untuk menentukan keberlakuan FD tersebut.

- SELECT kuliah.kode, kuliah.namakul, kuliah.sks, perkuliahan.nilai FROM kuliah NATURAL JOIN perkuliahan NATURAL JOIN mahasiswa WHERE mahasiswa.nim = 13514999 AND perkuliahan.nilai IS NOT null ORDER BY kuliah.sks DESC, kuliah.kode ASC;

Penjelasan : Kita ingin mendapatkan 4 data, dimana beberapa data tersebut ada duplikatnya di tabel lain (kode ada di tabel perkuliahan dan kuliah), maka harus dispesifikan selectnya dari tabel yang mana (kalo natural join sebenarnya gpp ga kasih tabel spesifik) , terus jangan lupa kasih syarat perkuliahan.nilai is not null biar kuliah yang diambil yang belum ada nilai ga dimasukin ke tabel hasil.

- SELECT kuliah.kode, kuliah.namakul, kuliah.sks, COUNT(perkuliahan.nim) FROM kuliah NATURAL JOIN perkuliahan WHERE perkuliahan.semester = 2 AND (perkuliahan.tahun = 2015 OR perkuliahan.tahun = 2016) GROUP BY kuliah.kode;

Penjelasan : sepertinya udah jelas :)

- Ada 2 cara :

1. Pake fungsi except, yaitu mengeliminasi semua row yang sama dari query di sebelah kiri

```
SELECT m.nama, m.prodi FROM mahasiswa AS m WHERE NOT EXISTS ((
SELECT kuliah.kode FROM kuliah WHERE kuliah.prodikul = m.prodi ) except (
SELECT p.kode FROM perkuliahan AS p WHERE p.nim = m.nim ));
```

Penjelasan : cari semua kode mata kuliah yang prodikul = prodi mahasiswa (query pertama), lalu cari semua kode mata kuliah yang telah diikuti mahasiswa(query kedua), kemudian pakai notasi NOT EXISTS dimana semua kode kuliah di prodi yang diikuti mahasiswa dikurangi(except) semua kode kuliah yang diikuti mahasiswa (kalau mahasiswa sudah ikut semua mata kuliah di prodinya, hasilnya harusnya kosong).

2. Pake count

```
SELECT m.nama, m.prodi FROM mahasiswa AS m WHERE (
SELECT COUNT(*) FROM perkuliahan NATURAL JOIN kuliah WHERE m.nim =
```

perkuliahan.nim AND m.prodi = kuliah.prodikul) = (
SELECT COUNT(*) FROM kuliah WHERE kuliah.prodikul = m.prodikul)

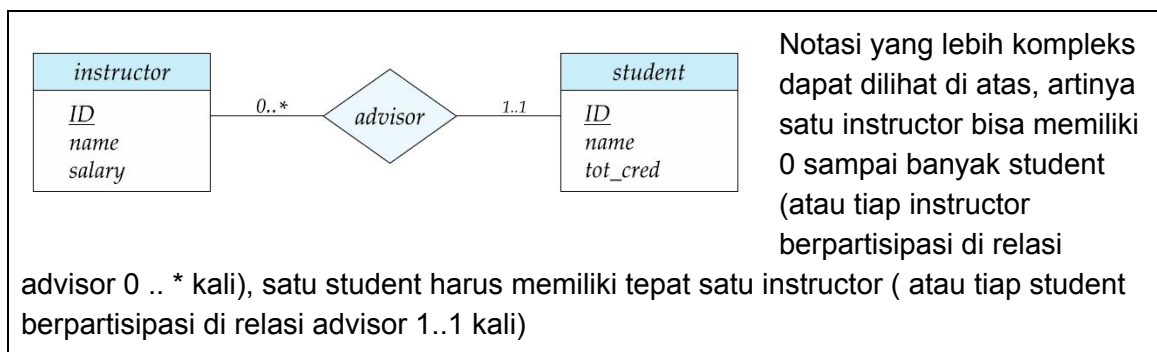
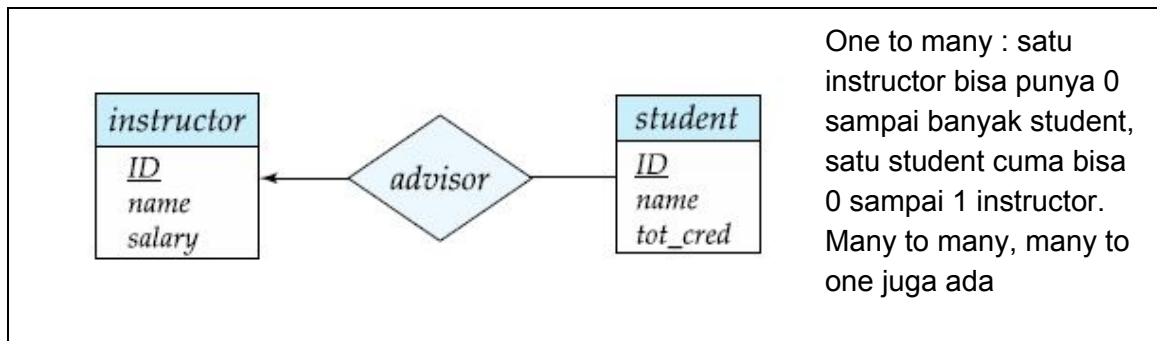
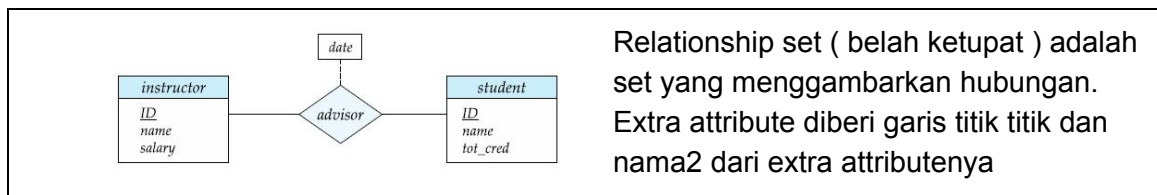
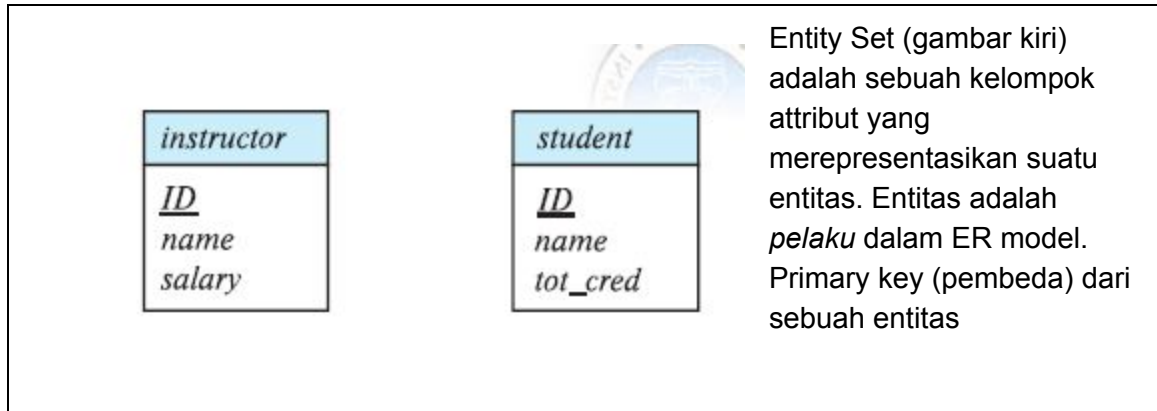
Penjelasan : pertama, hitung banyaknya kuliah dimana prodi mata kuliah = prodi mahasiswa (query terakhir) , lalu hitung banyaknya mata kuliah yang diikuti mahasiswa (query kedua), lalu sambungkan dengan query pertama

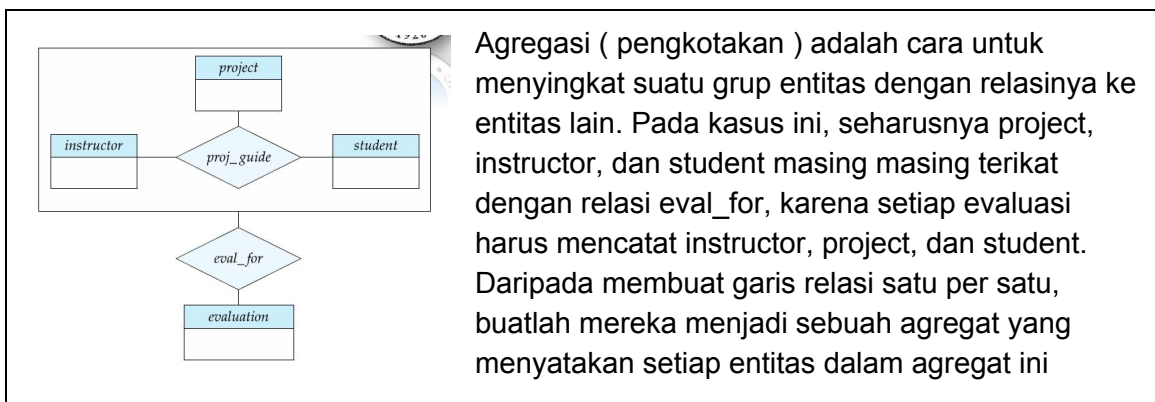
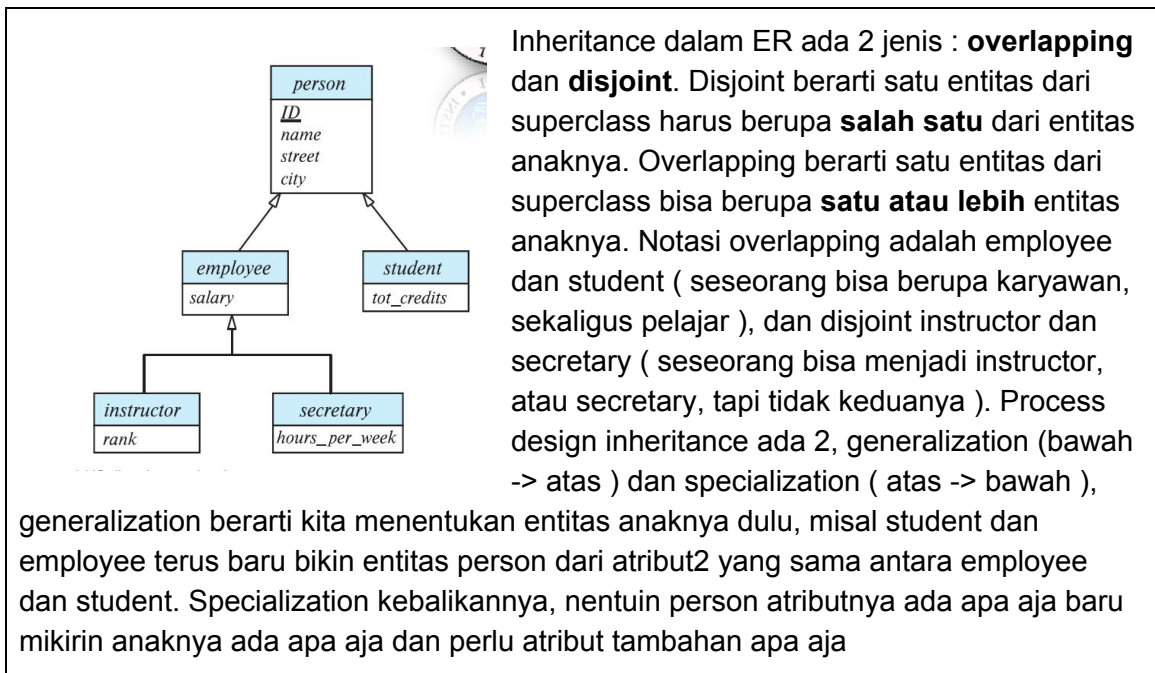
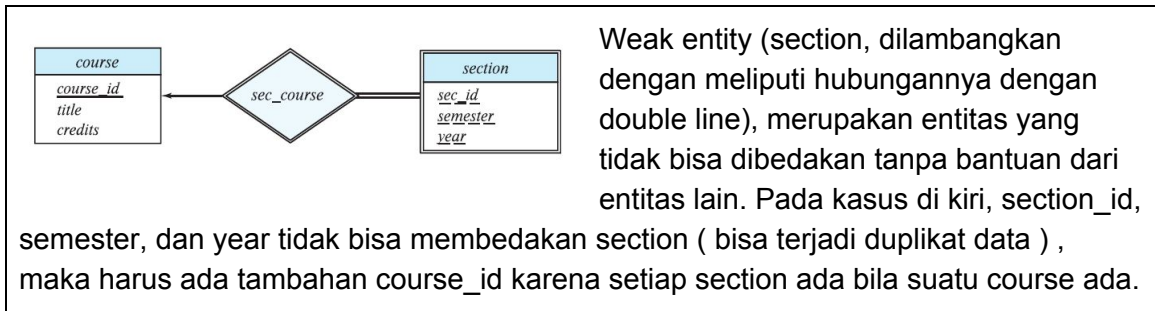
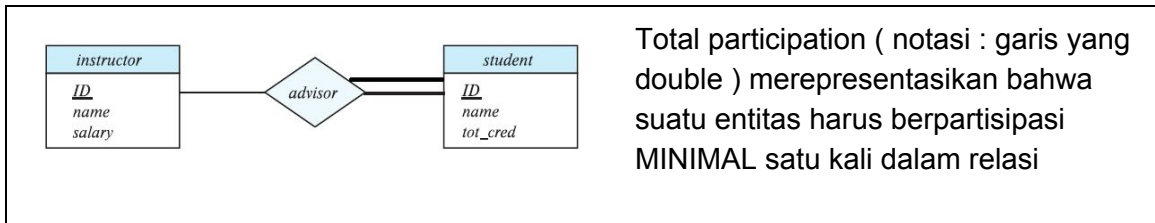
- d. SELECT * FROM (
SELECT COUNT(fakultas) as banyak_fakultas FROM (
SELECT DISTINCT prodi, fakultas FROM mahasiswa
) GROUP BY prodi)
WHERE banyak_fakultas > 1 ;

Penjelasan : Karena prodi → fakultas berarti prodi **unik**, maka seharusnya banyak fakultas tiap prodi tidak lebih besar dari 1, sehingga bila query yang men-*select* banyak fakultas lebih dari 1 mengembalikan suatu set, maka bisa dibilang functional dependency salah (karena prodi tidak menentukan suatu fakultas).

2) ER Model

ER Model adalah skema basis data yang merepresentasikan struktur logikal dari sebuah database.





terhubung dengan eval_for relation. Mohon bagi yg lebih ngerti agregat dan lebih bisa ngejelasin kegunaannya tambahin disini ya!

B. Transform ER to Relational Model

Website ini ngebantu banget, ada gambarannya :

<https://destiasalma.blogspot.com/2011/06/transformasi-model-er-ke-model-relasi.html>

Difficulty : Easy

Algoritma konversi ER ke Relational

- Mapping of Regular Entity Type
Convert normal saja, untuk atribut-atribut yang normal.
- Mapping of Weak Entity Type
Convert normal saja, tambahkan primary key dari strong entity.
- Mapping of Binary 1 1 Relationship
Kedua entity di convert normal saja
Tambahkan satu primary key di **salah satu** relasi (1).
- Mapping of Binary 1 N Relationship
Kedua entity di convert normal saja
Tambahkan foreign key pada relasi (N) berisi primary key dari relasi (1)
- Mapping of Binary M N Relationship
Kedua entity di convert normal saja
Relationship menjadi entity ketiga berisi primary key relasi (M) dan relasi (N)
- Mapping of Composite Attributes
Semua attribute composite di *flattened out* dengan membuat attribute terpisah untuk setiap komponen dari attribute composite ke dalam relasi.
Misal attribute composite : Name (first_name, middle_initial, last_name), address (street (street_num, street_name, apt_num), city, state, zip).
Maka isi relasi diubah menjadi {Primary key, first_name, middle_initial, , state, zip}
- Mapping of Multivalued Attributes
Jika terdapat multivalued dari suatu entitas (seperti no telp), setiap multivalued berdiri sendiri jadi 1 relasi
1 multivalued attribute = 1 relasi sendiri
Isi relasi terdiri atas {Primary Key, (Attribute multivalue)}
- Mapping of N ary Relationship
Kasus 1: Unary Relationship
 - Unary 1 N Relationship (One-to-Many)
Masukin foreign key ke diri sendiri (ID_Manager di relasi karyawan, dimana ID_Manager reference ke ID_Karyawan dari relasi karyawan yang sama)
 - *Unary M N Relationship (Many-to-Many)*
Entity convert ke relasi normal saja
Relationship isinya primary key dari relasi yang sama (course_id, preq_id)

Kasus 2: Ternary Relationship

- *Setiap entity (3 entity) berdiri sendiri jadi 3 relasi masing-masing*
 - *Relationshipnya dibuat jadi satu relasi sendiri, menyimpan **3 foreign key***
- Mapping of Derived Attributes
Langsung aja dibuang, karena akan ditampilkan dengan view.

Mapping ER Model Constructs to Relations

- Options for mapping specialization or generalization
Ada dua cara:
 1. Masing-masing entitas spesial dan general jadi relasi. Setiap entitas spesial bakal nyimpen primary key entitas general sebagai foreign key.
Trade off: nggak redundant, tapi bakal banyak join
 2. Yang dijadikan relasi cuma entitas spesialnya aja, masing-masing simpan semua atribut entitas general dan atribut entitas spesial.xzx
Trade off: nggak perlu banyak join, tapi jadi redundant. Terus, harus ada cara mastiin supaya atribut redundant konsisten di tiap relasi

Terus mending pake yang mana?

Buat nentuin cara mana yang lebih baik, harus dilihat dulu penggunaan atau sistemnya. Kalau seringnya pake data entitas spesial aja cukup cara 1; kalau sering butuh kombinasi data general x spesial, mending cara 2.

- Mapping of aggregation
Misalkan agregat terdiri atas 2 entitas atau lebih
Jika agregat A dihubungkan dengan suatu entitas B , maka:
 - Setiap entitas dalam agregat A, dijadikan sebagai relasi
 - Entitas B dijadikan sebagai relasi
 - Relationshipnya dijadikan sebagai relasi dengan ketentuan:
{Setiap primary key dari agregat A, primary key B}

C. Relational Database Design

Difficulty : Hard

Tips : Setiap bagian dalam Relational Database Design, langsung dicoba pada soal pada latihan mandiri yang sudah disediakan.

Credit : Michael Hans

1. Lossless Decomposition

Misalkan R merupakan relasi yang didekomposisi menjadi R1 dan R2.

Dekomposisi R1 dan R2 dikatakan lossless decomposition apabila memenuhi salah satu syarat berikut ini.

$$R1 \cap R2 \rightarrow R1$$

$$R1 \cap R2 \rightarrow R2$$

Bisa bilang mirip $(R1 \cap R2)^+ = R1$ atau $R2$ memanfaatkan FD yang kita punya

Bahasa mudahnya bila $(R1 \text{ natural join } R2) = R$, maka lossless

Bila $(R1 \text{ natural join } R2) \neq R$, maka lossy (misal abis dekomposisi, terus di join malah datanya nambah)

2. Functional Dependencies

Fungsi-fungsi yang menyatakan ketergantungan antara satu atribut dengan atribut lain.

Syarat : Domain harus unik, Kodomain bisa kemungkinan sama dari domain berbeda.

Misalnya: $R = \{A, B, C, D\}$

Functional Dependencies =

$A \rightarrow B$ (artinya B ditentukan oleh A)

$B \rightarrow C$ (artinya C ditentukan oleh B)

Notasi tambahan :

$A \rightarrow B, C$ (artinya A menentukan B, A menentukan C)

$A, B \rightarrow C$ (artinya A **dan** B menentukan C)

3. Penentuan Candidate Key dan Super Key

Ada dua cara dalam penentuan candidate key:

- Memanfaatkan functional dependencies
- Trial and error (not recommended)

Akan dijelaskan cara menentukan candidate key dengan FD. Karena candidate key adalah suatu key atau pasangan key yang bisa menentukan semua atribut dari suatu relasi, maka kita harus cari dengan FD, atribut atau pasangan atribut manakah yang bisa “menuju” ke semua atribut. Candidate key bisa lebih dari 1

Misalkan diketahui $R = \{A, B, C, D, E, F\}$ dengan himpunan FD adalah sebagai berikut.

$FD = \{A \rightarrow B, B \rightarrow C, E \rightarrow DF\}$

Penentuan candidate key dimulai dengan iterasi sebagai berikut:

- Key = $\{A, B, C, D, E, F\}$
Perhatikan bahwa DF ditentukan oleh E sehingga
- Key = $\{A, B, C, E\}$

Perhatikan bahwa C ditentukan oleh B sehingga

- Key = {A, B, E}

Perhatikan bahwa B ditentukan oleh A sehingga

- Key = {A, E}

Karena sudah dilakukan pengecekan pada masing-masing FD, maka candidate key yang berlaku adalah {A, E}

Super Key → Inheritance dari Candidate Key (tapi gak polymorphism ya)

Contoh : Beberapa super key yang relasi diatas adalah:

- AE
- ABE
- ADEF
- ACEF
- ABCE
- ABCDE

Referensi yutub orang india (ini bantu parah sih) :

<https://www.youtube.com/watch?v=vZCYWxE1bRg>

Bahasanya bahasa *boled* ya, gabungan inggris india wkwk

4. Axiom of Functional Dependencies (sangat useful)

Konsepnya sama persis dengan axiom pada Logika Komputasional yang lalu.

- Trivial = Reflexive Rule

Sesuatu yang udah obvious banget, ke dirinya sendiri. Ini **bukan rule**. FD $A \rightarrow B$ dikatakan **trivial** jika B merupakan bagian dari A.

- $A \rightarrow A$
- $B \rightarrow B$
- $AB \rightarrow AB$
- $AB \rightarrow A$ atau $AB \rightarrow B$

- **Reflexive rule (Armstrong Dasar)**

Sebenarnya sama seperti trivial, cuman konsepnya sebagai rule.

Kalau ada atribut {A} dan {B} adalah bagian dari {A,B}, maka berlaku

- $A \rightarrow A$
- $B \rightarrow B$
- $AB \rightarrow AB$
- $AB \rightarrow A$ atau $AB \rightarrow B$

- **Augmentation rule (Armstrong Dasar)**

Kiri nambah, kanan juga harus nambah dong.

- $A \rightarrow B$
- $AD \rightarrow BD$ (augmentation rule)

- **Transitivity rule (Armstrong Dasar)**

Prinsipnya kayak silogisme di Logika Komputasional.

- $A \rightarrow B$
- $B \rightarrow C$

- $A \rightarrow C$ (transitivity rule $A \rightarrow B$ dan $B \rightarrow C$)
- Union rule (Turunan)
Cara ngakalinnya dengan Aksioma Dasar:
 - $A \rightarrow B$
 - $A \rightarrow C$
 - $A \rightarrow AB$ (augmentation 1 dengan A)
 - $AB \rightarrow BC$ (augmentation 2 dengan B)
 - $A \rightarrow BC$ (transitivity 4 dan 5)

Prinsipnya penggabungan dua FD dengan domain yang sama

- $A \rightarrow B$
- $A \rightarrow C$
- $A \rightarrow BC$ (union rule 1 dan 2)

- Decomposition rule (Turunan)
Cara ngakalinnya dengan Aksioma Dasar:
 $AD \rightarrow BC$
 $BC \rightarrow B, BC \rightarrow C$ (Reflexive)
Jadi:
 $AD \rightarrow B, AD \rightarrow C$ (Transitivity)

Prinsipnya kalo kodomain bisa dipecah jadi dua dari suatu FD.

- $A \rightarrow BC$
- $A \rightarrow B$ (decomposition rule $A \rightarrow BC$)
- $A \rightarrow C$ (decomposition rule $A \rightarrow BC$)

Ingat! Yang didekomposisi hanya bagian kanan saja, ini paling dilarang

- $AD \rightarrow BC$
- $AD \rightarrow B, AD \rightarrow C$ (**BOLEH**)
- $A \rightarrow BC$ (**0 BESAR**)
- $D \rightarrow BC$ (**0 BESAR**)
- $A \rightarrow B$ (**Apalagi ini, 0 BESAR BANGET**)

- Pseudo-transitivity rule (turunan)
Cara ngakalinnya dengan Aksioma Dasar:
 $A \rightarrow B$
 $AD \rightarrow BD$ (Augmentation)
 $BD \rightarrow C$
Jadi
 $AD \rightarrow C$
Prinsipnya kayak substitusi variabel
 - $A \rightarrow B$
 - $BD \rightarrow C$

- $AD \rightarrow C$ (pseudo-transitivity rule $A \rightarrow B$ dan $BD \rightarrow C$)

5. Closure dari Himpunan FD

Konsepnya sama persis dengan closure pada Logika Komputasional yang lalu. Notasi closure F adalah F^+

Prerequisites : **Paham Axiom of Functional Dependencies**

Contoh :

Misalnya: $R = \{A, B, C, D\}$

Functional Dependencies = $\{A \rightarrow B, B \rightarrow CD\}$

Tentukan closure dari A^+ !

Jawab: $A \rightarrow B$ (functional dependency)

$A \rightarrow A$ (reflexive rule dari atribut A / trivial)

$A \rightarrow CD$ (transitivity rule $A \rightarrow B$ dan $B \rightarrow CD$)

$A \rightarrow ABCD$ (union rule $A \rightarrow B$, $A \rightarrow CD$, dan $A \rightarrow A$)

Jadi, closure dari $A^+ = A = AB = ABCD$.

Note: Sebenarnya bisa pake penomoran langkah biar memudahkan penulisan rule

Kapan sih closure ini digunakan? Banyak banget guys!

- Penentuan extraneous attribute
- Penentuan canonical cover
- Penentuan ekuivalensi dua buah himpunan FD

6. Normal Form

Basic Reading : <https://beginnersbook.com/2015/05/normalization-in-dbms/>

- First Normal Form (1NF)

Setiap values pada atribut dalam tabel bersifat atomik (tidak ada multivalued dalam satu entri data).

Hampir 99% relasi pasti sudah 1NF. Kalo tidak 1NF, harusnya relasi tersebut tidak bisa diimplementasikan dalam MySQL.

- Second Normal Form (2NF)

Suatu relasi R dengan FD F, memenuhi kaidah 2NF apabila untuk setiap atribut dalam relasi R:

1. Atribut adalah candidate key *atau* bagian dari suatu candidate key, atau
2. Atribut tidak partially dependent ke suatu candidate key

Partially dependent $\rightarrow$ ada atribut yang bergantung kepada sub-atribut dari candidate key.

Contoh: Diambil dari contoh (3) Penentuan candidate key

Misalkan diketahui $R = \{A, B, C, D, E, F\}$ dengan himpunan FD adalah sebagai berikut.

$FD = \{A \rightarrow B, B \rightarrow C, E \rightarrow DF\}$

Diketahui candidate key = $\{ (A,E) \}$

Akan tetapi, terdapat atribut yang **hanya** bisa diturunkan dari sub-candidate key. Atribut DF hanya bisa diturunkan dari E, atribut B dan C hanya bisa diturunkan dari A, sehingga relasi R tidak memenuhi 2NF. Note : syarat nomor 2 itu misalnya R(A,B,C,D,E) candidate key nya ada 3 , yaitu {(AB) , (AC), (AD)}, maka atribut A,B,C,D udah aman (adalah bagian dari suatu candidate key), misalnya E dependent ke AD, maka relasi sudah dalam 2NF.

- Third Normal Form (3NF)

Suatu relasi R dikatakan memenuhi prinsip 3NF, apabila :

- + Relasi R sudah dalam bentuk 2NF (setiap atribut bagian dari suatu candidate key atau setiap atribut tidak partially dependent ke suatu candidate key)
- + **Tidak ada** transitive dependency non-prime attribute pada super key
MAKSUDNYA : setiap atribut yang bukan bagian dari candidate key (non prime attribute) dependen ke suatu candidate key secara langsung (tidak ada transitive dependency) sebenarnya rule ini kalau diskip gapapa (rule dibawah udah mengcover rule ini) tapi rule ini untuk mempermudah dan mempercepat aja
- + Untuk setiap $X \rightarrow Y$ dalam F yang tidak trivial ($AB \rightarrow ABCD$ diubah dulu jadi $AB \rightarrow CD$), memenuhi **salah satu** dari:
 - + X adalah super key
 - + Y adalah prime attribute

Note: prime attribute = atribut bagian dari candidate key

Super key = inheritance dari candidate key

Non-prime attribute = atribut SELAIN candidate key

Apa sih transitive dependency itu?

Kalo candidate key = {A} dan terdapat FD dimana:

$A \rightarrow B$

$B \rightarrow CD$

Disini, A adalah super key, namun CD **transitive dependency** terhadap A.

Contoh: <https://beginnersbook.com/2015/05/normalization-in-dbms/>

- Boyce-Codd Normal Form (BCNF)

Versi strict dari 3NF, strict nya dimana?:

- + Untuk setiap $X \rightarrow Y$ dalam F, X wajib **super key**.

Cenderung lebih mudah ditentukan, cukup dicek apakah LHS dari setiap FD apakah merupakan super key atau bukan.

7. Tujuan Normalisasi

Setelah dicek bahwa suatu relasi R dengan himpunan FD tidak berada dalam normal tertinggi (good form), maka diperlukan **dekomposisi** dengan syarat:

- Setiap relasi hasil dekomposisi dalam keadaan good form.

- Setiap relasi hasil dekomposisi harus lossless decomposition.
- Setiap relasi hasil dekomposisi harus dependency preserving.

Arti dependency preserving adalah bila semua FD dari relasi hasil dekomposisi digabungkan, akan sama dengan FD relasi awal ($F_1 \cup F_2 \cup \dots \cup F_n$)⁺ = F⁺

8. Extraneous Attribute

Suatu atribut X dalam FD F dikatakan **extraneous** apabila tidak mengubah F⁺.

Jenis extraneous untuk FD $\alpha \rightarrow \beta$ dapat dibagi menjadi 2, yaitu:

- Extraneous LHS (left hand side)
Atribut A extraneous dalam α misalnya F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$
- Extraneous RHS (right hand side)
Atribut A extraneous dalam β misalnya $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha \rightarrow (\beta - A))\}$ logically implies F
Notasi ini boleh dihafal, tp akhir"nya nanti gamungkin ngecek semua atribut dalam FD, biasanya tergantung feeling aja jadinya

Sebagai contoh:

Diketahui suatu relasi R dengan FD = {AB → CD, A → C}

Perhatikan bahwa atribut **C adalah extraneous pada AB → CD**. Jika C dihapus, maka AB → D dengan closure dari AB⁺ = ABCD (gunakan teknik closure) tetaplah sama seperti sebelumnya.

Bisa diperoleh AB → C juga dari AB → D dengan memanfaatkan axiom dengan A → C.

9. Canonical Cover

Suatu functional dependencies F dikatakan memenuhi canonical cover apabila:

- F dapat mewakili Fc
- Fc dapat mewakili F
- Tidak ada atribut extraneous pada Fc
- LHS dari setiap FD pada Fc bernilai unik dan tidak redundan.

Contoh kasus:

R = {A, B, C}

F = {A → BC, B → C, A → B, AB → C}

+ Step 1: Hilangkan redundant LHS

Perhatikan bahwa terdapat redundant LHS pada A → BC dan A → B sehingga akan dilakukan penggabungan A → BC dan A → B menjadi A → BC

F1 = {A → BC, B → C, AB → C}

+ Step 2: Hilangkan atribut extraneous

A merupakan atribut extraneous pada AB → C.

Cek apakah menghilangkan A akan mempengaruhi closure AB.

Ternyata, sudah terdapat $B \rightarrow C$ sehingga penghilangan A tidak mengubah closure dari fungsi tersebut.

Maka yang awalnya $AB \rightarrow C$ menjadi $B \rightarrow C$, namun sudah dalam set.

$F2 = \{A \rightarrow BC, B \rightarrow C\}$

C merupakan atribut extraneous pada $A \rightarrow BC$

Cek apakah menghilangkan C akan mempengaruhi closure A

Ternyata, dengan transitivity rule pada $A \rightarrow B$ dan $B \rightarrow C$, diperoleh $A \rightarrow C$ sehingga penghilangan C tidak mempengaruhi closure fungsi tersebut.

Maka, yang awalnya $A \rightarrow BC$ berubah menjadi $A \rightarrow B$.

$F3 = \{A \rightarrow B, B \rightarrow C\}$

Jadi, Canonical cover $F_c = \{A \rightarrow B, B \rightarrow C\}$

- F_c bisa mewakili F
- F bisa mewakili F_c
- Setiap LHS sudah unik (check)
- Tidak ada atribut extraneous (check)

10. Ekuivalensi Dua Buah Himpunan FD

Prerequisites: Fasih dalam penggunaan closure

Misalkan terdapat relasi R dengan dua buah himpunan FD, yaitu F dan G.

Untuk mengecek apakah dua buah Himpunan FD ekuivalen, algoritmanya cukup sederhana, yaitu:

- Pengecekan F adalah cover dari G.
 - Untuk setiap FD dalam G, tentukan closure dari setiap LHS dalam F.
 - Kemudian cek apakah subset closure tersebut dapat diperoleh pada salah satu FD dalam G.
 - Jika terdapat kasus yang gagal. Stop.
 - Jika semua kasus berhasil. Success.
- Pengecekan G adalah cover dari F.
 - Untuk setiap FD dalam F, tentukan closure dari setiap LHS dalam G.
 - Kemudian cek apakah subset closure tersebut dapat diperoleh pada salah satu FD dalam F.
 - Jika terdapat kasus yang gagal. Stop.
 - Jika semua kasus berhasil. Success.
- Jika dua pengecekan tersebut sukses, maka $F \equiv G$.

11. Dependency Preservation

Misalkan R adalah sebuah relasi yang didekomposisi menjadi R1 dan R2 dengan FD adalah F.

FD dalam F dapat dicek dari hasil dekomposisi **tanpa harus melakukan R1 join R2.**

Contoh:

Misalkan $R = \{A, B, C, D\}$

Dengan hasil dekomposisi adalah $R_1 = \{A, B, C\}$ dan $R_2 = \{B, D\}$

Dengan functional dependencies $F = \{A \rightarrow BC, B \rightarrow D\}$

Perhatikan bahwa $A \rightarrow BC$ dapat dicek pada relasi R_1 .

Perhatikan bahwa $B \rightarrow D$ dapat dicek pada relasi R_2 .

Maka, hasil dekomposisi memenuhi prinsip dependency preservation.

12. Praktek Normalisasi & Dekomposisi

Saran: kalo misalkan emang ambigu, langsung liat latman collection aja ya

Berikut ini adalah **Five Magical Steps (FMS)** untuk melakukan praktek normalisasi dan dekomposisi:

- 1) Tentukan candidate key dari relasi R dengan memanfaatkan FD.
 - 2) Lakukan pengecekan bentuk normal tertinggi dari suatu relasi R dengan FD berupa F. Pengecekan normalisasi **selalu** dimulai dari bentuk tertinggi dengan urutan adalah $BCNF \rightarrow 3NF \rightarrow 2NF$
 - 3) Jika relasi sudah dalam bentuk BCNF, tidak perlu lakukan normalisasi
 - 4) Jika relasi belum dalam bentuk BCNF, lakukan dekomposisi sesuai dengan bentuk normal yang diperoleh sebelumnya. Berikut adalah langkah dekomposisi sesuai dengan bentuk normal tertinggi R yang berlaku.
 - + 1NF : pisahkan FD yang menyebabkan partial-dependent, LHS bukan key.
 - + 2NF : pisahkan semua transitive dependency non-prime attribute ke relasi baru. (misal $F = \{A \rightarrow B, B \rightarrow C\}$ C transversal dependency ke A)
 - + 3NF : pisahkan FD yang menyebabkan masalah ke relasi baru.
- Semua pemisahan dilakukan ke dalam **dua** dekomposisi, yaitu R_1 dan R_2 dimana:
- + R_1 : relasi R yang dipisahkan berdasarkan pemisahan 2NF / 3NF diatas
 - + R_2 : relasi R sisanya ($R - R_1$)
- Kedua dekomposisi dijamin harus lossless decomposition dan dependency preservation. Kalo tidak dependency preserving, maka gagal decompose.
- 5) Kembali ke langkah (1) untuk pengecekan R_1 dan R_2 .

Mari kita langsung saja menerapkan **FMS** pada soal berikut. Cekidot!

(Diambil dari Soal Latihan Mandiri 17-03-2020)

Diberikan sebuah relasi $R = \{A, B, C, D, E, H\}$ dengan $F = \{A \rightarrow CD, E \rightarrow AH\}$.

Tentukan bentuk normal tertinggi dari relasi R dan lakukan dekomposisi bila relasi R belum dalam bentuk normal tertinggi.

Jawaban:

Candidate key:

- Key = $\{A, B, C, D, E, H\}$

Perhatikan bahwa CD ditentukan oleh A sehingga

- Key = {A, B, E, H}
Perhatikan bahwa AH ditentukan oleh E sehingga
- Key = {B, E}
Karena sudah dilakukan pengecekan pada masing-masing FD, maka candidate key yang berlaku adalah **{(B, E)}**

Pengecekan bentuk normal tertinggi:

- BCNF
Relasi R tidak dalam bentuk BCNF karena terdapat FD dimana LHS bukan super key (A bukan super key pada $A \rightarrow CD$).
- 3NF
Relasi R tidak dalam bentuk 3NF karena terdapat transitive dependency terhadap super key, yaitu $E \rightarrow A$ dan $A \rightarrow CD$.
- 2NF
Relasi R tidak dalam bentuk 2NF karena AH partial dependent ke {(B, E)}.

Karena relasi R dalam bentuk 1NF, dilakukan dekomposisi dengan metode sesuai bentuk normalnya (dalam hal ini 1NF):

- R1 = {A, C, D, E, H} dengan F1 = { $A \rightarrow CD$, $E \rightarrow AH$ } dan candidate key = {E}
- R2 = {B, E} dengan F2 = {} dengan candidate key = {(B, E)}

Lakukan pengecekan terhadap R1 dan R2 dengan langkah yang sama.

1) Untuk relasi R1= {A, C, D, E, H}

Candidate key = {E}

Pengecekan bentuk normal tertinggi:

- BCNF
Relasi R tidak dalam bentuk BCNF karena terdapat FD dimana LHS bukan super key (A bukan super key pada $A \rightarrow CD$).urutan adalah BCNF $\rightarrow$ 3NF $\rightarrow$ 2NF
- 3NF
Relasi R tidak dalam bentuk 3NF karena terdapat transitive dependency terhadap super key, yaitu $E \rightarrow A$ dan $A \rightarrow CD$.
- 2NF
Relasi R berada dalam bentuk 2NF karena semuanya fully-dependent

Karena relasi R1 dalam bentuk 2NF, dilakukan dekomposisi dengan metode sesuai bentuk normalnya (dalam hal ini 2NF):

- R11 = {A, C, D} dengan F11 = { $A \rightarrow CD$ } dan candidate key = {A}
- R12 = {E, A, H} dengan F12 = { $E \rightarrow AH$ } dengan candidate key = {E}

1.1) Untuk relasi R11 = {A, C, D}

Candidate key = {A}

Pengecekan bentuk normal tertinggi:

BCNF $\Rightarrow$ Sudah dalam bentuk BCNF karena setiap FD memiliki LHS super key

1.2) Untuk relasi R12 = {E, A, H}

Candidate key = {E}

Pengecekan bentuk normal tertinggi:

BCNF $\Rightarrow$ Sudah dalam bentuk BCNF karena setiap FD memiliki LHS super key

2) Untuk relasi R2 = {B, E}

Candidate key = {B, E}

Pengecekan bentuk normal tertinggi:

BCNF $\Rightarrow$ Sudah dalam bentuk BCNF karena setiap FD memiliki LHS super key
(dalam kasus ini, tidak ada FD, jadi aman-aman saja)

Jadi, hasil dekomposisi R agar diperoleh bentuk normal tertinggi adalah:

R11 = {A, C, D} dengan F11 = {A $\rightarrow$ CD}

R12 = {E, A, H} dengan F12 = {E $\rightarrow$ AH}

R2 = {B, E} dengan F2 = {}

Note: ***Alangkah indahnya bila tidak ada FD dalam R, ga bakal ribet-ribet ada normalisasi dan dekomposisi, seperti R2***

D. Integrity Constraint

Difficulty : Medium

Credit : Annisa Ayu Pramesti

1. Pendahuluan

Integrity Constraint merujuk kepada **konsistensi** dari suatu basis data.

Contoh integrity constraint yang biasa dikenal adalah sebagai berikut.

- Checking account harus memiliki saldo lebih besar dari \$10.000 USD
- Gaji karyawan bank memiliki nilai setidaknya \$4.00 per jam.
- Seorang pelanggan harus memiliki nomor telepon yang tidak NULL

2. Klasifikasi Constraint

Klasifikasi constraint dibagi menjadi 4 constraint, yaitu:

- Type constraint

Adalah constraint yang mengikat sebuah tipe data tertentu. Umumnya constraint ini digunakan ketika dibuat sebuah **tipe data baru**, seperti **INTPOS**, yaitu nilai $INT > 0$.

- Attribute constraint

Adalah constraint yang mengikat atribut saja. Contohnya adalah suatu atribut A memiliki tipe INTEGER. Teknik menjaga integritas atribut dapat dilakukan dengan membuat user defined type lalu menerapkannya pada atribut yang bersangkutan.

User defined type adalah seperti **INTPOS**. Dalam kasus attribute constraint, contohnya adalah seperti mendeklarasikan sebuah atribut HargaBarang dengan tipe data **INTPOS**.

- Relation constraint

Adalah constraint yang mengikat suatu relasi. Teknik menjaga integritas relasi dapat dilakukan check constraint pake CHECK clause pada saat CREATE atau ALTER TABLE, memberikan constraint NOT NULL atau UNIQUE pada atribut yang diinginkan pada tabel tersebut. **Perhatikan** bahwa constraint diberikan kepada atribut tetapi klasifikasi constraint adalah relation constraint (mengacu pada solusi latihan integrity constraint). UNIQUE, PRIMARY KEY, CHECK, masuknya relation constraint.

- Database constraint

Adalah constraint yang mengikat database artinya pengecekan constraint nya melibatkan lebih dari satu relasi. Teknik menjaga integritas database contohnya menggunakan TRIGGER clause (dapat menggunakan CHECK clause dengan subquery tetapi tidak disarankan karena hampir tidak ada DBMS yang mengimplementasikan [correlated] subquery untuk check constraint). Foreign Key Reference juga termasuk bagian dalam database constraint.

Gimana sih cara menentukan klasifikasi constraint dari suatu kasus?

Perhatikan potongan percakapan berikut ini.

Type Constraint : Ah, aku mau buat tipe baru ah, namanya A, yaitu integer > 0.
 Attribute Constraint : Hey, aku deklarasikan atribut B dengan tipe data A ya.
 Relation Constraint : Hey kalian, inget kalau setiap atribut B kudu unik di satu relasi
 Database Constraint : Weh, gua numpang pinjem atribut B (foreign key) di relasi lain ya

Secara formal, berikut ini kasus-kasus yang saling berkaitan satu sama lain.

- Sebuah bobot penilaian dengan nilai antara A, B, C, dan D akan dibuat dalam tipe baru yang namanya **Indeks**. (**Type Constraint**)
- **Nilai Akhir** dideklarasikan dengan tipe **Indeks**. (**Attribute Constraint**)
- **Nilai Akhir** ini akan menentukan jumlah **SKS** yang akan diambil mahasiswa dalam satu relasi Mahasiswa. (**Relation Constraint**)
- **Nilai Akhir** akan **direfer** oleh tabel perwalian mahasiswa dan dosen (**Database Constraint**)

3. Integrity Constraint in MySQL

Integrity Constraints dalam Sebuah Relasi(semua ini relation constraint):

- Not Null
 Deklarasi suatu atribut agar tidak boleh Null.
 Name **varchar(20) not null**
 Budget **numeric(12,2) not null**
- Primary Key
 Deklarasi atribut yang menjadi primary key dari suatu relasi.
 ID CHAR(8) NOT NULL **PRIMARY KEY**
 Jika primary key terdiri dari beberapa atribut maka:
PRIMARY KEY (nama, wilayah)
- Unique
Unique (A₁, A₂, .., A_m) menyatakan bahwa A₁, A₂, .. A_m membentuk candidate key.
- Check
Check (P) dimana P adalah predikat
 Check (gender in ("Female", "Male"))
 Check (age > 0)
 Syntax di MySQL: **CONSTRAINT nama_constraint CHECK (P)**

4. Referential Integrity

Intinya yang jadi reference harus ada

Dideklarasikan dalam FOREIGN KEY clause, contoh:

foreign key (dept_name)
 references department
on delete set null

on update cascade;

Perhatikan bahwa on delete dan on update tidak mesti sama.

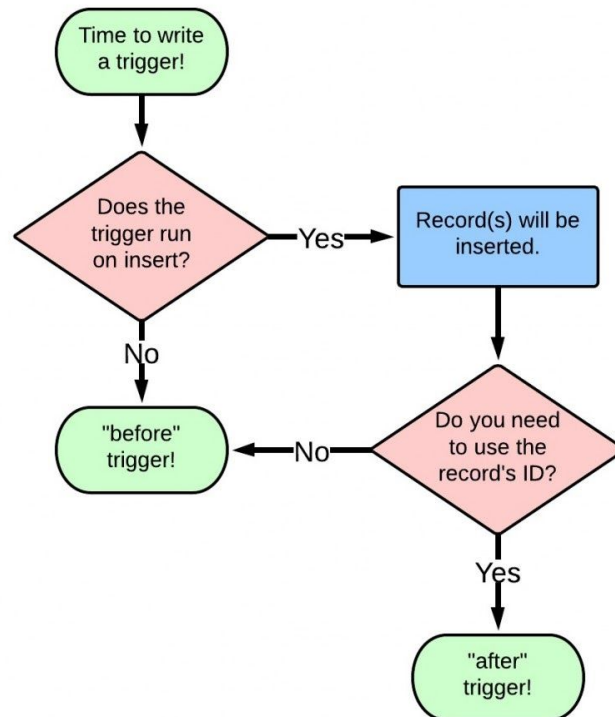
- Cascade
Kalau parent dihapus maka semua anaknya juga dihapus (no orphans are allowed). Kalo di on update, misal parent di update, anaknya juga ke update.
- Set null
Kalau parent dihapus/di update maka atribut yang direferensi oleh anaknya akan menjadi null. Untuk menggunakan ini maka atribut yang menjadi **foreign key** tidak boleh diberi constraint **not null**.
- Set default
Sama seperti set null bedanya dia berubah menjadi nilai defaultnya (jika tidak dideklarasikan defaultnya maka null).
- No action / Restrict
Penghapusan/ update parent tidak diperbolehkan jika dia memiliki anak.

Untuk melakukan insert tanpa melanggar referential integrity:

- Insert parentnya dulu semua baru anaknya
- Insert anaknya langsung tapi untuk atribut yg dijadikan foreign key di set null dan diupdate setelah parent di insert (hanya berlaku jika tidak ada constraints not null)
- Menggunakan trigger untuk mengupdate tabel parent

5. Trigger

- Triggering event : **insert, update, delete**
- Bisa dilakukan sebelum / sesudah : **before, after**
- Kapan menggunakan before? Ketika sebelum event terjadi mau di cek sudah valid atau belum (sesuai kondisi yang diinginkan atau nggak)
- Kapan menggunakan after? Ketika setelah event terjadi, tuple tersebut digunakan untuk mengupdate tuple di tabel lain



- **referencing new row as** : digunakan ketika insert atau update
- **referencing old row as** : digunakan ketika delete atau update
- **begin atomic** digunakan ketika sejumlah perubahan perlu dilakukan pada shared variable dan penting untuk mencegah instance lain mengeksekusi statement tsb
- Level trigger: **for each row, for each statement**
<https://www.geeksforgeeks.org/difference-between-row-level-and-statement-level-triggers/>
- Setelah statement **begin .. end** selalu diakhiri semicolon (kecuali end else)
- Statement yang berada di dalam begin end merupakan satu klausa penuh sehingga perlu diberi semicolon
- **declare** untuk mendeklarasikan variabel. Contoh: **declare** nama_variabel tipe;
- **declare** selalu diakhiri semicolon (merupakan statement utuh)
- contoh:
create trigger credits_earned **after update of** takes on (grade)
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
 and (orow.grade = 'F' or orow.grade is null)
begin atomic
 update student
 set tot_cred= tot_cred +
 (select credits

```

        from course
        where course.course_id= nrow.course_id)
    where student.id = nrow.id; ← perhatikan semicolon
end;

```

create trigger Barang_after_update_trigger **after update of** Barang **on** (hargadasar)

```

    referencing new row as nrow
    for each row
    begin
        if (nrow.hargadasar is not null) then
            begin
                UPDATE Pesanan
                SET totalharga = harga * nrow.kuantitas
                WHERE IDBarang = nrow.IDBarang; ← perhatikan semicolon
            end else ← hargadasar null, tidak ada semicolon
            begin
                UPDATE Pesanan
                SET totalharga = null
                WHERE IDBarang = nrow.IDBarang; ← perhatikan semicolon
            end;
        end;
    end;

```

create trigger stmt_level_trigger
after update of Highschooler
for each statement ← perhatikan bahwa memakai statement
begin
 update Highschooler
 set grade = NULL
 where grade = 13;
end;

create trigger row_level_trigger
after update of Highschooler
referencing new row as nrow
for each row ← perhatikan bahwa memakai row
when nrow.grade = 13
begin
 UPDATE Highschooler
 SET grade = NULL
 WHERE ID = nrow.ID;
End;

6. Function

- Mirip trigger tp ga di invoke tiap event trigger, buat nge run harus direfer oleh statement lain misal: SELECT nama_function(param)
- **returns** untuk menentukan tipe return value
- **return** untuk mengembalikan suatu nilai
- Terdapat **declare, begin .. end** sama seperti trigger
- contoh:

```

create function dept_count (dept_name varchar(20))
returns integer ← return type dideklarasikan terlebih dahulu
begin
    declare d_count integer; ← perhatikan semicolon
    select count ( * ) into d_count
    from instructor
    where instructor.dept_name = dept_name
    return d_count;
End;

```
- contoh pemanggilan:

```

select dept_name, budget
from department
where dept_count (dept_name ) > 12 ← memanggil fungsi dept_count

```

E. Storage & File Structure

Difficulty : Medium

Metode : Seperti Quiz OS K-3, diambil dari soal UAS Basis Data 2019.

1. Dalam konteks magnetic disk, jelaskan apa yang dimaksud dengan:
 - a. **Access time**
Waktu yang dibutuhkan untuk melakukan request write/read pada disk ketika perpindahan / transfer data dimulai.
 - b. **Seek time**
Waktu yang dibutuhkan untuk mengunjungi suatu track sesuai dengan track yang diinginkan.
 - c. **Rotational latency**
Waktu yang dibutuhkan untuk mengembalikan sector dari disk yang diperlukan dengan melakukan rotasi terhadap suatu piringan disk. Umumnya $\frac{1}{2} \times \text{rotation speed}$
 - d. **Data-transfer rate**
Besarnya ukuran data rata-rata baik dalam proses pengambilan data atau penyimpanan data.
 - e. **Mean time of failure**
Ekspektasi waktu rata-rata dari suatu disk untuk berjalan secara berkelanjutan tanpa adanya kegagalan atau error.

Note : ini review materi OS yang sudah diujikan sebelumnya.

2. Sebutkan dan jelaskan dengan singkat 7 parameter kinerja untuk organisasi file
 - R : jumlah storage yang diperlukan untuk sebuah record.
 - T_F : waktu yang dibutuhkan untuk memperoleh record sesuai keinginan dari file.
 - T_N : waktu yang dibutuhkan untuk mendapatkan record selanjutnya dalam suatu file.
 - T_I : waktu yang dibutuhkan untuk mengupdate file dengan insert record.
 - T_U : waktu yang dibutuhkan untuk mengupdate file dengan mengganti record.
 - T_X : waktu yang dibutuhkan untuk pembacaan keseluruhan isi file (exhaustive).
 - T_Y : waktu yang dibutuhkan untuk melakukan reorganisasi file.

Jembatan Kelayak : **R**ecord, **F**etch, **N**ext, **I**nsert, **U**ppdate, **E**xhaustive Search, **Y**ow Reorganize!

Alternatif: r y u fine? = **r**ecord, **y**ow reorganize, **u**ppdate, **f**etch, **i**nsert, **n**ext, **e**xhaustive search

3. Jelaskan bagaimana mekanisme kerja atau cara penyisipan record pada file yang diorganisasi secara:
 - a. **Hashing**

Record-record di dalam file ditempatkan berdasarkan hasil perhitungan terhadap sejumlah atribut record yang akan menentukan pada blok data mana record akan ditempatkan.

b. Heap

Record-record di dalam file ditempatkan berdasarkan urutan kedatangan.

c. Multitable Clustering

Record-record dari relasi yang berbeda dapat disimpan pada file yang sama.

d. Sequential

Record-record di dalam file ditempatkan berdasarkan keterurutan atribut kunci pencarian (search-key) dari masing-masing record.

4. Berikan 3 perbandingan / perbedaan antara sparse index dan dense index. Perbandingan dapat meliputi: struktur / cara kerja indeks, atau kelebihan dan kekurangannya.

Sparse Index	Dense Index
Hanya mengandung index record untuk beberapa nilai search-key saja.	Index record muncul untuk setiap nilai dari search-key dalam file.
Membutuhkan space yang lebih kecil dan maintenance yang lebih sedikit terkait insertion dan deletion.	Membutuhkan space yang lebih banyak dan maintenance yang lebih banyak terkait insertion dan deletion
Umumnya lebih lambat dibandingkan dense index dalam mengunjungi suatu record.	Umumnya lebih cepat dalam pencarian suatu record.
Sparse index dengan sebuah index entry untuk setiap block dalam file, berkorespondensi dengan nilai search-key yang lebih sedikit dalam block.	Dense index akan mengembalikan keseluruhan nilai search-key yang berkorespondensi dengan indeks tersebut.

F. Index & Schema Tuning

Difficulty : Relative

Indexing

Terdapat dua jenis index dalam basis data, yaitu:

- Ordered Indices, search key disimpan dalam susunan teratur
- Hash Indices, search key terdistribusi secara uniform pada bucket dengan menggunakan hash function.

Parameter Performa Index

- Access types supported efficiently, seperti record dengan nilai yang spesifik atau rentang yang spesifik.
- Access time
- Insertion time
- Deletion time
- Space overhead

Primary Index dan Secondary Index

Berikut ini adalah perbandingan penggunaan primary index dan secondary index.

Primary Index	Secondary Index
Dalam sequential ordered file, index ini memiliki search key yang disusun dalam keadaan teratur dalam file	index ini memiliki search key yang urutannya berbeda dengan keterurutan dalam sebuah file.
Index record langsung menunjuk kepada record yang ingin dituju sesuai dengan nilai dari search-key.	Index record tidak langsung menunjuk kepada suatu bucket berisi data, melainkan menunjuk ke suatu bucket yang berisi pointer menuju record yang sebenarnya sesuai dengan nilai dari search-key.
Pemindaian teratur lebih efisien	Pemindaian teratur lebih mahal

Multilevel Index

- Jika primary index tidak muat dalam memory, pengaksesan menjadi lebih mahal.
- Maka dari itu, primary index tetap disimpan dalam disk sebagai sequential file dan buat sparse index di dalamnya.
 - + Outer index $\Rightarrow$ sparse index dari primary index
 - + Inner index $\Rightarrow$ primary index dari file
- Jika outer index juga terlalu besar untuk muat dalam memory, tinggal tambahkan saja level index berikutnya dan seterusnya.

- Indeks dalam semua level **harus terupdate** baik dalam insertion dan deletion dari file.

Index Update: Insertion

- Dense index: Kalau search key belum terdapat di index, tambahkan search key ke index
- Sparse index:
 - Kalau data yang dimasukkan bukan blok data baru, gak usah nambahin apa-apa
 - Kalau data yang dimasukkan ngebuat blok data baru, tambahkan search key pertama pada blok data tersebut ke index

Index Update: Deletion

- Dense index: Kalau search key mau didelete, tinggal delete aja selo
- Sparse index:
 - Kalau data terkait sparse index sudah dihapus semua, maka index juga dihapus

Hash Index

- Peletakan data-data dalam index disusun berdasarkan hash function $h(v) = a$
- Misalkan hash function $h(v)$ dimana v adalah search-key, maka $h(v)$ merupakan posisi bucket dalam index (yaitu posisi ke- a bila $h(v) = a$)

Praktek Indexing

Penggunaan index dalam MySQL cukup sederhana. Mengingat penggunaan index ini mengikuti prinsip index dalam pencarian kata pada buku, maka index dalam MySQL pun juga demikian. Praktek Indexing ini bertujuan untuk **memperoleh performansi pencarian query** yang lebih **efektif** dan **lebih cepat**. Semua adalah tentang **Waktu**.

Misalkan terdapat relasi Person = {firstName, lastName, birthday}

1. Untuk menambahkan index pencarian dalam MySQL, cukup gunakan syntax sederhana sebagai berikut.

```
CREATE INDEX person_firstName_idx ON person (firstName);
```

Maka, pencarian yang berkaitan dengan menggunakan **firstName** akan mengembalikan output query yang **jauh lebih cepat** dibandingkan dengan tanpa membuat index.

2. Kita juga bisa membuat index yang melibatkan dua kolom sekaligus pada kondisi pencarian yang mungkin lebih sering digunakan, contohnya adalah fullName, yaitu firstName dan lastName.

```
CREATE INDEX person_firstName_lastName_idx ON person
(firstName, lastName);
```

Maka, pencarian yang berkaitan dengan menggunakan **firstName** dan **lastName** akan mengembalikan output query yang **jauh lebih cepat** dibandingkan dengan tanpa membuat index gabungan tersebut.

Jika kita mencari data Person dengan FirstName = "Faris-kun", maka kecepatan eksekusi menggunakan index akan **jauh lebih cepat** dibandingkan tanpa indeks.

Tapi perlu diingat, bahwa index **memakan storage** (belum lagi waktu mengupdate index setiap ada perubahan pada tabel terkait) sehingga pembuatan index juga perlu dipikirkan dengan baik-baik, tidak semua atribut perlu dibuatkan indeksinya. Sesuaikan aja sama workload yang memang sering digunakan (mirip Schema Tuning).

Jadi, perbedaan utama antara basis data dengan Index Tuning dan bukan adalah "Hanya membuat indeks saja."

Untuk penggunaan indexing dalam MySQL lebih lanjut, dapat dicek video berikut ini.

Source : <https://www.youtube.com/watch?v=fsG1XaZEa78>

Fyi, ketika membuat tabel mysql biasanya tabel primary dan foreign key sudah otomatis terindex oleh SQL saat pertama dibuat. Sebenarnya indexing banyak jenisnya tapi sepertinya untuk di basdat cukup tahu saja indexing itu seperti apa dan cara pembuatannya... lanjutannya bakal di MBD :D

Database Tuning (Overview):

Kegiatan untuk **mengatur** (*tuning*) agar sebuah database bekerja lebih cepat.

Hal-hal yang mempengaruhi *performance* sebuah database:

- Hardware (Storage, CPU, RAM, Network)
- Pengaturan OS-nya
- Parameter server DB
- Database design / schema (Schema Tuning dan Indexing)
- Index yang ada di DB (Indexing)
- Query yang diberikan ke DB

Kunci dari konsep Schema Tuning ini adalah memahami **workload**.

Workload ini meliputi:

- Query yang sering digunakan
- Update yang sering dilakukan

Dari sana, bisa dianalisis; **relasi** apa yang terkait, **atribut** apa yang terkait, relasi mana saja yang dilakukan **join**, dsb. Dari situlah, kita tau **bagian** mana yang harus dilakukan **schema tuning**.

Schema Tuning:

Melakukan perubahan terhadap schema untuk mempercepat query dsb, dicapai lewat beberapa hal:

- Splitting Table (memecah relasi), bisa lewat dua cara:
 - Secara horizontal (memecah record)

Kapan makenyaa?

 - a. Jika tabel terlalu besar
 - b. Jika hanya beberapa records saja yang biasanya diproses / digunakan
 - Secara vertikal (memecah atribut)

Makenya kapan bro?

 - a. Saat beberapa kolom atau atribut diakses lebih sering daripada atribut lainnya dalam satu tabel

(Secara general, *splitting table bikin programnya kompleks*)
- Melakukan denormalisasi:
 - Menambah redundant kolom

-> Untuk menghilangkan seringnya penggunaan join di berbagai queries
 - Menambah atribut turunan
 - Collapsing tables

-> if most users need to see the full set of joined data from tables

-> Jadinya 2 tabel di collapse trus dijadiin satu tabel
 - Duplicating tables

-> if a group of users regularly needs only a subset of data
- Menggabungkan record yang sering dilakukan join menjadi satu (*clustering together*)

Database Development Process

SDLC

-> System Development Life Cycle (Siklus kehidupan pengembangan sistem)

- a. Positif: Detil, terencana, jelas
- b. bad bruah: Makan waktu, bikinnya lama

Prototyping

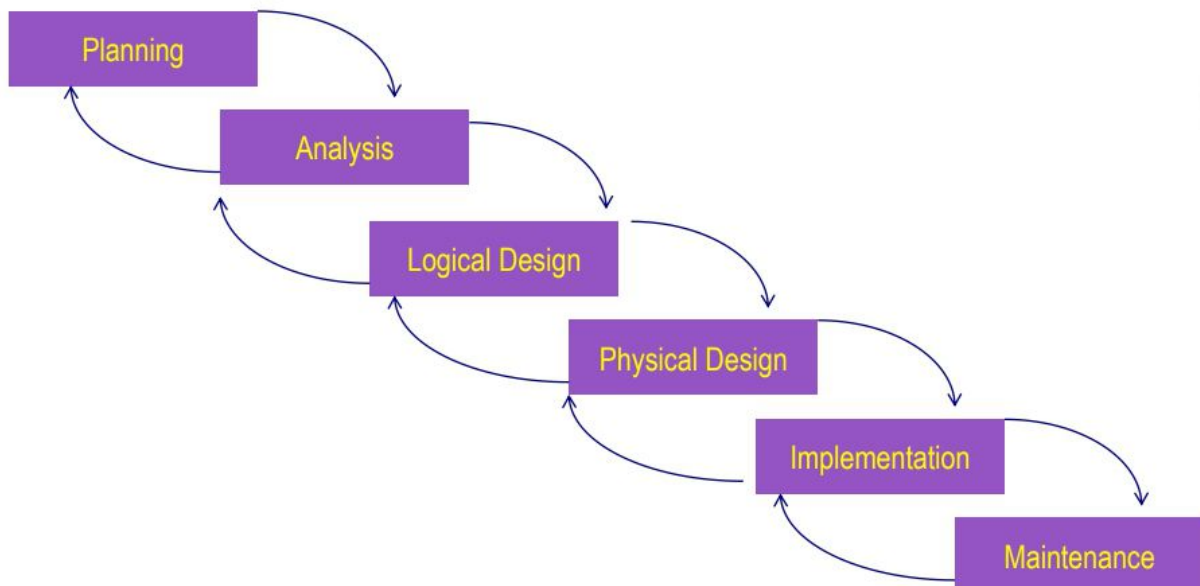
-> RAD/Rapid application development (pengembangan aplikasi cepat)

-> Bikin model data (ERD dll) singkat

- a. Positif: Db didefinisi pas bikin prototipe awal (faster)
- b. Bad bruah: Implementasi dan maintenance ulang tiap prototype

Siklus SDLC:

Systems Development Life Cycle



a. Planning

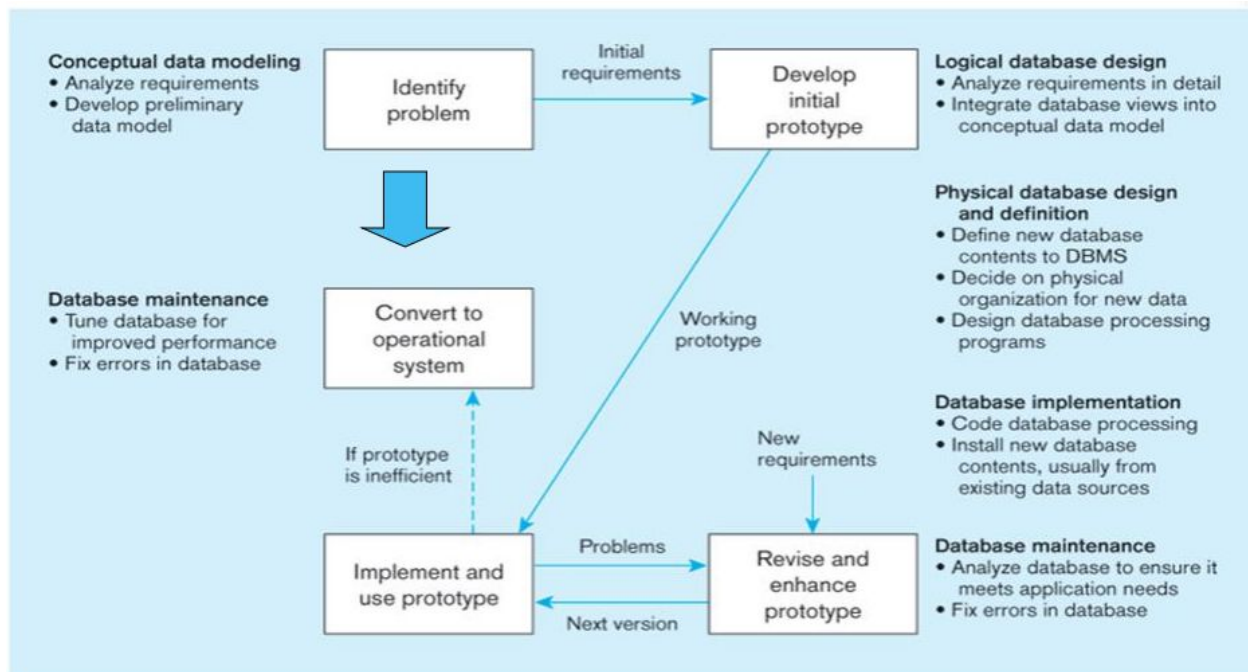
- Tujuan: Ngerti masalah
- Deliverable: Daftar hal yang perlu dianalisis
- Aktifitas: Model konsep awal (ERD) & model perusahaan

b. Analisis

- Tujuan: Analisis kebutuhan secara rinci (welcome skpl :P)

- Deliverable: Spek fungsional sistem (Kebutuhan klien secara rinci, kayak pas milestone 1)
 - Aktifitas: Model konsep rinci (ERD direvisi :P)
- c. Logical design
- Tujuan: Ubah model konseptual ke logical (relasional, hierarchical dll. Kalo di sql relasional aja)
 - Deliverable: Spek desain detail - diagram relasional + constraint, index, dll yang perlu
 - Aktifitas: bikin diagram relasional
- d. Physical design
- Tujuan: Menentukan implementasi fisiknya mau kayak gimana entar
 - Deliverable: Struktur datanya - Format record, besar blok, cara akses, security, backup, recovery, dll (belum diajarin caranya di kuliah, baru teorinya dikit)
 - Aktifitas: Desain fisik basis data
- e. Implementasi
- Tujuan: implementasi hasil desain ke basis data, pengetesan, bikin dokumentasi, training
 - Deliverable: Basis data yang dah operasional, dokumentasi, materi buat training
 - Aktifitas: implementasi basis data, program, dokumentasi, instalasi
- f. Maintenance
- Tujuan: Monitoring, memperbaiki, bikin lebih bagus
 - Deliverable: Audit berkala (intinya hasil cek berkala)
 - Aktifitas: Pemeliharaan basis data, analisis performa, tuning biar lebih bagus, betulin error

Teknis prototyping:



- a. Identify problem:
 - Analisis kebutuhan
 - Bikin ERD

Kalo ada kebutuhan awal
VVV

- b. Develop initial prototype: (bikin prototype awal)
 - Analisis kebutuhan secara detail
 - Logical design (Bikin desain relasional)
 - Physical design (Bikin desain fisik - hubungannya sama materi storage, cara akses, security, backup & recovery, dll)

Kalo dah ada prototype yang bisa jalan
VVV

- c. Implement and use prototype
 - Bikin kode programnya
 - Implementasi database
 - Analisis database - performanya, sudah sesuai spek belom

Kalo ada error ^^^

VVV Versi baru

d. Revise and enhance prototype

- Betulin error
- Menambahkan yang belum sesuai spek (atau kalau spek nambah kek OOP)
- Memperbaiki performa

Kalo cuman prototype gak cukup (mesti dites live)

VVV

e. Convert to operational system

- Tuning buat performance
- Fix error